Rebasing Instruction Prefetching: An Industry Perspective

Yasuo Ishii, Jaekyu Lee, *Member, IEEE*, Krishnendra Nathella, *Member, IEEE*, and Dam Sunwoo, *Member, IEEE*

Abstract—Instruction prefetching can play a pivotal role in improving the performance of workloads with large instruction footprints and frequent, costly frontend stalls. In particular, Fetch Directed Prefetching (FDP) is an effective technique to mitigate frontend stalls since it leverages existing branch prediction resources in a processor and incurs very little hardware overhead. Modern processors have been trending towards provisioning more frontend resources, and this bodes well for FDP as it requires these resources to be effective. However, recent academic research has been using outdated and less than optimal frontend baselines that employ smaller structures, which may result in misleading outcomes. In this letter, we present a detailed FDP microarchitecture and evaluate two improvements, better branch history management and post-fetch correction. We believe that our FDP-based frontend design can serve as a new reference baseline for instruction prefetching research to bridge the gap between academia and industry.

Index Terms—Instruction prefetching, branch predictor

1 Introduction

The instruction footprint of both data-center and client applications has been growing larger, causing more instruction cache (I-cache) misses and frequent pipeline stalls in modern processors. Instruction prefetching [1], [2], [3], [4], [5], [6] is a well-known technique to reduce I-cache misses. One of these techniques is Fetch-Directed Prefetching (FDP) [2], which leverages existing branch prediction components, such as direction predictors [7] and the Branch Target Buffer (BTB), that are highly accurate and well provisioned in modern processors, making FDP effective with only marginal hardware overhead. The predicted instruction addresses are stored in a dedicated structure called a Fetch Target Queue (FTQ). I-cache lines that FTQ entries point at are fetched into the I-cache before needed, effectively decoupling the frontend from the rest of the pipeline. The decoupled frontend enables the processor to run ahead and predict future instruction streams independent of instruction fetch. Recently disclosed commercial CPU designs [8], [9], [10], [11] indicate that they employed a variant of FDP.

FDP relies on the BTB to sufficiently cover the control flow. Since a BTB hit is the only way to identify a branch instruction at the prediction stage, a taken branch that misses in the BTB goes undetected even if the branch direction predictor could have predicted the branch outcome correctly. Thus, to cover more branches and improve the effectiveness of FDP, the industry has been trending towards adding more resources in the branch predictor, as seen in recent CPUs [9], [10], [11], [12].

Academic research, however, shows the opposite trend. As shown in Table 1, many academic papers used a smaller BTB for their evaluations, while commercial processors, based on recently disclosed information, implemented a larger BTB. While some of these papers, such as Shotgun [5] and Boomerang [6], proposed BTB prefetching mechanisms built on top of FDP to compensate for limited BTB capacities, they require complex basic-block-based BTB designs. Also, software instruction prefetching mechanisms, such as AsmDB [13], used a simple frontend design without FDP.

In the recently held 1st Instruction Prefetching Championship (IPC-1) [3], the baseline had a shallow 12-instruction FTQ (limited run-ahead) and perfect target prediction (unrealistic). While designing a realistic BTB within the given budget was allowed, because of insufficient branch information available under the rules, it was difficult to implement an effective FDP. Using such a sub-optimal baseline and restricted framework may result in misleading research outcome.

To understand the gap between industry and academic baselines, we compared various instruction prefetching mechanisms, including a next-line, the top-3 prefetchers (`D-JOLT`, `FNL+MMA`, and `EIP`) from IPC-1, and perfect [15] with and without FDP. Note that having sufficiently large 192-instruction FTQ effectively enables a simple FDP capability [2]. All mechanisms use the same framework with perfect target prediction, which was used in IPC-1. Fig. 1 shows the speedup over the baseline (no prefetching, no FDP). As presented in IPC-1, the top-3 prefetchers offer over 28 percent speedup over the baseline, achieving close to perfect prefetching (30.6 percent). We see that the simplistic FDP shows a 30.2 percent performance improvement. The top-3 IPC-1 prefetchers with FDP provide little benefit over FDP alone. These results show that 1) the baseline used in academia lags behind that of industry and 2) employing additional prefetching mechanisms on top of FDP cannot be justified without reasonably high performance benefits.

In this paper, we present an effective FDP design with comprehensive microarchitecture details, which is more representative of a modern CPU. We also evaluate two features to further improve FDP: 1) using taken-only branch target history that makes FDP more tolerant of BTB-miss not-taken branches, which also improves the BTB utilization, and 2) a post-fetch correction scheme that eagerly re-steers the prefetched instruction stream upon detecting BTB misses of taken branches.

This paper claims the following key contributions:

- We present a comprehensive microarchitecture of the FDP design and evaluate two features that improve FDP.
- We show that our FDP outperforms other prefetching mechanisms for public IPC-1 traces and adding a dedicated prefetcher on top of FDP provides marginal performance improvement.
- Our improved FDP design will bridge the gap in frontend designs between industry and academia and can be used as the baseline for future instruction prefetching research.

2 Improving Fetch Directed Prefetching

FDP enables a cost-efficient, high-performance frontend, but its performance can be further improved by two enhancements, better branch history management and post-fetch correction. We describe our FDP design with these features in this section.

2.1 Branch History Management

Branch predictors rely on the recent branch history for accurate predictions. Branch outcomes are collected after instructions are fetched [18] and maintained in the global history register (GHR). We can update the GHR using either directions from *all* branches (e.g., Eq. (1)) or targets from *taken* branches (e.g., Eq. (3)).

$$direction\_history = (direction\_history \ll 1) \mid branch\_taken \quad (1)$$

$$target\_hash = (instruction\_address \gg 2) \oplus (target \gg 3) \quad (2)$$

$$target\_history = (target\_history \ll 2) \oplus target\_hash. \quad (3)$$

• *The authors are with Arm, Austin, TX 78735.*
E-mail: {yasuo.ishii, jaekyu.lee, krishnendra.nathella, dam.sunwoo}@arm.com.

Manuscript received 10 Sept. 2020; revised 7 Oct. 2020; accepted 25 Oct. 2020. Date of publication 30 Oct. 2020; date of current version 11 Dec. 2020.
(Corresponding author: Krishnendra Nathella.)
Recommended for acceptance by K. Inoue.
Digital Object Identifier no. 10.1109/LCA.2020.3035068

TABLE 1
BTB Capacity Gap Between Academia and Industry

Academia	BTB	Industry	BTB
Shotgun [5]	2.1K-entry	AMD Zen2 [8]	7K-entry
Confluence [4]	1.5K-entry	Samsung Exynos M3 [10]	16K-entry
Divide&Conquer [14]	2K-entry	Arm Neoverse N1 [9]	6K-entry

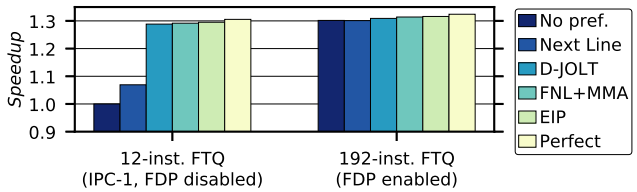

Fig. 1. Prefetching limit study using the IPC-1 framework.

The direction history is widely used in academic research. However, precise direction history can be disturbed by the run-ahead capability enabled by FDP, in particular when not-taken branches miss in the BTB. To make accurate branch prediction, the GHR needs continuous updates with all branch outcomes. However, if a branch misses in the BTB, it goes undetected, and the GHR is not updated. While we can correct the GHR for taken branches after branches get resolved, not-taken branches cannot get this opportunity, leading to an imprecise GHR.

BTB-miss not-taken branches can be handled in two ways. First, we can skip the GHR update for these branches, but this may result in imperfect history, potentially leading to more branch mispredictions. Second, the GHR can be corrected once the branch has been identified after prediction. This scheme incurs more frontend flushes and backend pipeline stalls because the GHR correction may change the branch predictor output. In Section 3.3, we compare the different history management policies to see how they affect performance.

On the other hand, taken-only branch target history naturally resolves the imprecise branch history issue since not-taken branches are ignored for the GHR update. This strategy also allows the BTB to store only taken branches. These benefits led the industry to employ the target branch history [12], [17], [18]. Based on this observation, we adopt it for our FDP design as well.

2.2 Post-Fetch Correction

In the traditional FDP implementation, on a BTB miss, a branch may go undetected, and no prediction is made. This would lead to fetching wrong I-cache lines for taken branches until the branch misprediction is detected. Assuming that an undetected branch is not-taken is a valid approach, which was adopted in some commercial processors [8]. However, this works well only if the instruction footprint fits in the BTB. Otherwise, FDP will frequently go down the sequential fetch flow, causing more mispredictions due to BTB capacity misses.

However, we have observed that modern branch predictors [7] can correctly predict the direction of these BTB-miss branches. Based on this observation, we propose to add a post-fetch correction (PFC) capability to FDP . The idea is to unconditionally make direction predictions on all instructions (including non-branch instructions), as in the Alpha EV8 processor [16], and store the prediction information (direction and BTB hit/miss) in the FTQ. When instructions are decoded after the fetch stage, we can accurately identify branch instructions and their targets.[1] Then, the FTQ is checked whether 1) an unconditional branch instruction was predicted not taken or missed in the BTB, or 2) a conditional branch was predicted taken and missed in the BTB. When either condition is met, the FTQ is flushed, the GHR is fixed, and the prefetch stream resumes from the new branch target. Otherwise, I-cache lines will be fetched from the sequential path. By eagerly detecting BTB-miss taken branches and fixing FTQ, PFC can prevent I-cache pollution and improve performance.

1. Branch targets can be calculated for PC-relative branches (offset embedded in an instruction) and function returns (target obtained from RAS), but indirect branches need to wait for branch resolution.

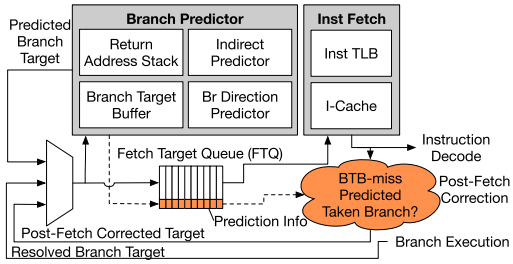

Fig. 2. The overview of frontend microarchitecture design.

2.3 Comprehensive Frontend Design for FDP

Fig. 2 shows an overview of our frontend design. Similar to existing FDP designs, the frontend contains separate branch prediction and instruction fetch pipelines. The branch predictor comprises the BTB, branch direction predictor, indirect predictor, and return address stack (RAS). It predicts the next instruction address, which is enqueued in the FTQ and looked up in the I-cache. On a miss, a cache-fill request is initiated.

FTQ is the only additional structure that FDP requires. We assume fixed-length instructions, but FDP can handle variable instruction lengths. Each FTQ entry covers a 32-byte *aligned* instruction block,[2] and its state is updated when instruction translation lookaside buffer (I-TLB) or I-cache tag lookup is done. Each FTQ entry has a 48-bit *instruction start address*, an 1-bit *predicted taken* to indicate whether the block is terminated by a predicted taken branch, a 3-bit *offset* for the instruction end address (by a taken branch), an 8-bit *branch direction hint*, and a 2-bit *state* to track I-TLB/I-cache tag lookup completion. FTQ need not store physical address since an I-cache tag lookup follows immediately after an I-TLB lookup. Our FDP uses a 24-entry FTQ from an empirical study, and its storage overhead of 186 bytes.

3 Evaluation

3.1 Methodology

We evaluate our proposed FDP in Champsim [19], an open-source trace-based simulator that was used for IPC-1. We run 50M instructions with 50M warmup using traces published by IPC-1 [3]. Traces include various server, client, and SPEC [20] workloads, and each trace shows over 5 percent performance uplift with a perfect I-cache over a 32KB I-cache. We implemented a comprehensive frontend microarchitecture with various branch predictors, including the BTB, indirect, and RAS predictors.

We use core parameters similar to Intel Sunny Cove [21]. The branch predictors are sized based on recently announced commercial processors [8], [9], [10] and tuned for evaluated traces. We use a 260-bit branch history length for both TAGE and ITTAGE. The branch history is updated by targets from taken branches, as shown in Eq. (3). The branch predictor bandwidth is double the fetch bandwidth to support the run-ahead capability [9]. Using 2-entry (16-instruction) FTQ disables FDP because of the limited run-ahead capability. For comparison, we evaluated the following mechanisms using common parameters shown in Table 2:

- Baseline: no FDP, no prefetching.
- Next line (`NL1`): prefetch the next line upon a cache miss.

2. One FTQ entry covers up to 8 instructions.

TABLE 2
Simulation Parameters

Parameter	Value
Out-of-order Core	6-wide, 24-entry (192-instruction) FTQ, 60-entry Decode Queue, 352-entry Re-order Buffer, 128-entry Reservation Station
L1 BTB	128-entry, 2-way, 1-cycle latency
L2 BTB	8192-entry, 4-way, 2-cycle latency
Branch predictors	TAGE-SC [7]: 8-table, 1024-entry/table, ITTAGE [25]: 8-table, 512-entry/table, 32-entry RAS
Caches	64B block, I-cache: 32KB, 8-way, 16 MSHRs; L1 data: 48KB, 12-way, 16 MSHRs, Next-Line Prefetcher; L2: 512KB, 8-way, 32 MSHRs, L3: 2MB, 16-way, 64 MSHRs, Signature Path Prefetcher [26]

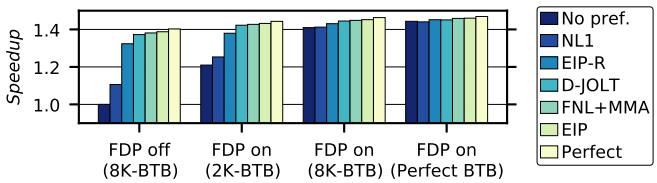

Fig. 3. IPC improvement by instruction prefetching.

- The top-3 prefetchers in IPC-1: `FNL+MMA` [22], `D-JOLT` [23], `EIP` (with the original 128KB 34-way Entangled Tables) [24], and `EIP-R` (with a realistic 27KB 8-way Entangled Table).
- Perfect prefetching (`Perfect`): proposed in [15], where we assume that the prefetch brings the data into the cache instantaneously but still sends out the request to the memory subsystem to simulate the traffic overhead.
- FDP: our improved FDP with 24-entry (192-instruction) FTQ. PFC is enabled. FDP can be employed independent of other configurations.

3.2 IPC Improvement

We evaluate prefetching mechanisms with and without FDP under different BTB capacities. Fig. 3 shows speedup over the baseline. Without FDP, `NL1` and `EIP-R` offer 10.6 and 32.4 percent speedup, respectively. Three IPC-1 prefetchers, `D-JOLT`, `FNL+MMA`, and `EIP`, show 37.2, 38.1, and 38.8 percent speedup, respectively. Their results are close to `Perfect` (40.3 percent). FDP alone realizes a 41.0 percent speedup thanks to 1) instruction prefetches by FDP and 2) BTB access latency hidden by using FTQ. If the FTQ contains enough pending transactions, the pipeline can tolerate some bubbles caused by BTB access latency. FDP outperforms both `EIP-R` and `EIP` by 8.6 and 2.2 percent, respectively.

FDP shows significantly better performance over the baseline, and additional performance can be obtained in two ways: better branch target prediction and instruction prefetching. As explained in Section 2.1, BTB misses limit the effectiveness of FDP. We identify that the perfect BTB improves the performance of FDP by 3.4 percent, which also shows the upper bound of BTB prefetching [4], [5], [6]. On the other hand, better instruction prefetching shows slightly higher performance uplifts. FDP with `EIP` and `Perfect` give 4.3 and 5.4 percent additional performance improvement, respectively.

As explained in Section 1, FDP requires enough BTB capacity to be effective. FDP with a 2K-entry BTB result is much less effective than with a 8K-entry BTB (21.0 versus 41.0 percent). Other mechanisms overcome this deficiency by prefetching.

3.3 Performance Impact of Branch History Organization

Many modern processors use taken-only branch target history [18], but we notice that academic papers generally use other configurations. To see the performance impact of history management schemes, we evaluate configurations with different history types, history corrections, and BTB allocation policies, as shown in Table 3. Note that we selected 280-bit branch direction history from an empirical study using the idealized GHR configuration to achieve the best branch prediction accuracy.

TABLE 3
Branch History Management Policies

Configuration	Type	Not-taken branch history correction	BTB allocation
FDP (THR)	Target	Not necessary	Taken-only
Idealized GHR (Ideal)	Direction	Fix history w/o flush	Taken-only
GHR config 0 (GHR0)	Direction	Do not fix	Taken-only
GHR config 1 (GHR1)	Direction	Do not fix	All
GHR config 2 (GHR2)	Direction	Fix history and flush	Taken-only
GHR config 3 (GHR3)	Direction	Fix history and flush	All

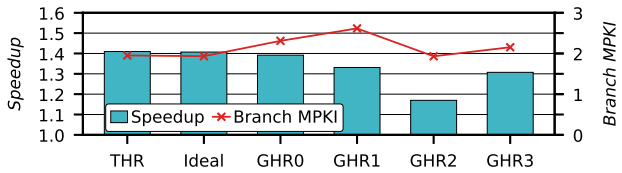

Fig. 4. Branch direction history management results.

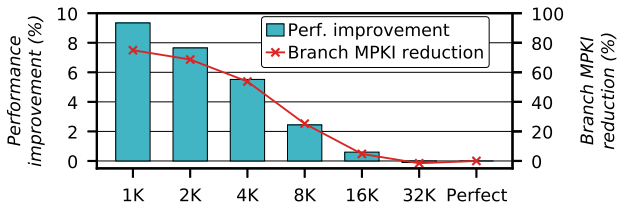

Fig. 5. Performance improvement and branch misprediction reduction achieved by enabling PFC for different BTB sizes.

Fig. 4 shows the performance and branch mispredictions per kilo instruction (MPKI) results. `THR` shows similar performance to `Ideal`, showing the benefit of using branch target history. Compared to `Ideal`, `GHR2` shows similar branch prediction accuracy, but frontend flushes to fix up the history incur more pipeline stalls, resulting in 23.7 percent lower performance. Not fixing the history as in `GHR0` increases branch mispredictions by 19.5 with 1.5 percent lower performance than `Ideal`. The taken-only BTB allocation policies show better prediction accuracy (`GHR0` versus `GHR1` and `GHR2` versus `GHR3`), but the performance impact varies depending on whether we correct not-taken branch histories. This experiment shows that the FDP configuration with the taken-only branch target history outperforms all other configurations, and thus this should be used in the frontend design.

3.4 Performance Impact of Post-Fetch Correction

PFC can improve the effectiveness of FDP, particularly with smaller BTBs. To confirm this, we evaluate PFC with BTB sizes varying from 1K to 32K entries by increasing the number of sets. Fig. 5 shows that PFC results in 2.4 and 9.3 percent better performance than when PFC is not enabled for an 8K-entry BTB and an 1K-entry BTB, respectively, but only 0.1 percent for a 32K-entry BTB.

Fig. 5 also shows how PFC affects the number of branch mispredictions. With an 8K-entry BTB, PFC reduces misprediction by 25.2 percent. With smaller BTBs, PFC can significantly improve the BTB performance, resulting in 75 percent misprediction reduction for an 1K-entry BTB. However, PFC with a 32K-entry BTB increases mispredictions by 1.5 percent because of strongly biased

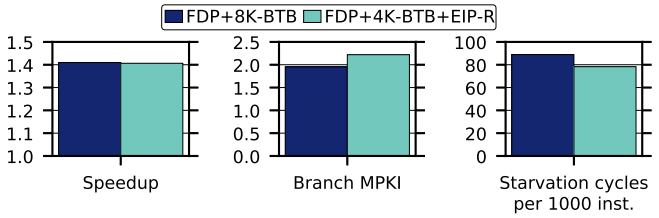

Fig. 6. ISO-budget analysis.

conditional branches. For example, any *never-taken* branch will not be allocated in the BTB while the large BTB has all other previously taken branches. When a direction predictor makes a wrong prediction (taken) for this branch with a BTB miss, PFC incorrectly fixes the FTQ, re-streeing the instruction stream to the wrong taken path and eventually leading to a pipeline flush.

3.5 ISO-Budget Comparison

Dedicated instruction prefetchers offer some performance improvement on top of FDP (Section 3.2), but these prefetching mechanisms require additional storage and complex logic. Based on Exynos M3 data [10], [11], we estimated that a BTB entry needs 7 bytes per branch. Since `EIP-R` (Section 3.1) has similar storage cost as a 4K-entry BTB, we compare 1) 8K-entry BTB and 2) 4K-entry BTB with `EIP-R`, on top of FDP.

Fig. 6 shows speedup, branch MPKI, and starvation cycles per kilo instructions. Here, *starvation* is due to fetch-stalls and *Starvation Cycles* is defined as the total number of cycles when the decode-queue contains less than a decode-width number of instructions. We find that thanks to a larger BTB, FDP with an 8K-entry BTB has 12 percent fewer branch mispredictions. However, timely prefetches by `EIP-R` enable it to achieve 13.5 percent lower starvation cycles. Although the two mechanisms take different approaches, we find that both perform similarly (41.0 versus 40.6 percent speedup). This result suggests that the complexity in having an additional prefetching mechanism cannot be justified over the straightforward design of FDP with a larger BTB.

4 Conclusion

Addressing frontend performance bottlenecks is critical for applications with large code footprints. However, academic and industrial communities have diverged and are using different approaches to resolve the same issue. We revisited FDP that many commercial CPUs have employed, presented a comprehensive design, and evaluated some features to address challenges. Our FDP design offers 41.0 percent speedup over no prefetching with a marginal 186-byte hardware overhead, which outperforms IPC-1 winners and is 5.4 percent lower than perfect prefetching. This design can also be coupled with other prefetchers to further improve performance for more frontend intensive workloads. We believe that our work can guide future instruction prefetching research to be more impactful to commercial CPUs.

Acknowledgments

The authors would like to thank the anonymous reviewers, Akanksha Jain, Calvin Lin, Alex Rico, Jose Joao, and Yonghae Kim for their feedback to improve the paper.

References

[1] M. Ferdman, C. Kaynak, and B. Falsafi, "Proactive instruction fetch," in *Proc. 44th Annu. IEEE/ACM Int. Symp. Microarchit.*, 2011, pp. 152–162. [Online]. Available: https://doi.org/10.1145/2155620.2155638
[2] G. Reinman, B. Calder, and T. Austin, "Fetch directed instruction prefetching," in *Proc. 32nd Annu. IEEE/ACM Int. Symp. Microarchit.*, 1999, pp. 16–27.
[3] IPC-1, "The 1st instruction prefetching championship," 2020. [Online]. Available: https://research.ece.ncsu.edu/ipc/
[4] C. Kaynak, B. Grot, and B. Falsafi, "Confluence: Unified instruction supply for scale-out servers," in *Proc. 48th Annu. IEEE/ACM Int. Symp. Microarchit.*, 2015, pp. 166–177. [Online]. Available: https://doi.org/10.1145/2830772.2830785
[5] R. Kumar, B. Grot, and V. Nagarajan, "Blasting through the front-end bottleneck with shotgun," in *Proc. 23rd Int. Conf. Archit. Support Program. Lang. Operating Syst.*, 2018, pp. 30–42.
[6] R. Kumar, C. Huang, B. Grot, and V. Nagarajan, "Boomerang: A metadata-free architecture for control flow delivery," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit.*, 2017, pp. 493–504.
[7] A. Seznec, "TAGE-SC-L branch predictors again," *Championship Branch Prediction-5*, 2014.
[8] D. Suggs, M. Subramony, and D. Bouvier, "The AMD "Zen 2" processor," *IEEE Micro*, vol. 40, no. 2, pp. 45–52, Mar./Apr. 2020.
[9] A. Pellegrini *et al.*, "The Arm Neoverse N1 platform: Building blocks for the next-gen cloud-to-edge infrastructure SoC," *IEEE Micro*, vol. 40, no. 2, pp. 53–62, Mar./Apr. 2020.
[10] J. Rupley, "Samsung Exynos M3 processor," *IEEE Hot Chips 30 Symp.*, 2018.
[11] B. Grayson *et al.*, "Evolution of the Samsung Exynos CPU Microarchitecture," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit.*, 2020, pp. 40–51.
[12] N. Adiga, J. Bonanno, A. Collura, M. Heizmann, B. R. Prasky, and A. Saporito, "The IBM z15 high frequency mainframe branch predictor," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit.*, 2020, pp. 27–39.
[13] G. Ayers *et al.*, "AsmDB: Understanding and mitigating front-end stalls in warehouse-scale computers," in *Proc. 46th Int. Symp. Comput. Archit.*, 2019, pp. 462–473.
[14] A. Ansari, P. Lotfi-Kamran, and H. Sarbazi-Azad, "Divide and conquer frontend bottleneck," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit.*, 2020, pp. 65–78.
[15] P. Michaud, "PIPS: Prefetching instructions with probabilistic scouts," *1st Instruction Prefetching Championship*, 2020.
[16] A. Seznec, S. Felix, V. Krishnan, and Y. Sazeides, "Design tradeoffs for the alpha EV8 conditional branch predictor," in *Proc. 29th Annu. Int. Symp. Comput. Archit.*, 2002, pp. 295–306.
[17] V. Uzelac and A. Milenkovic, "Experiment flows and microbenchmarks for reverse engineering of branch predictor structures," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2009, pp. 207–217.
[18] AMD, "Software optimization guide for amd family 17h processors," 2017. [Online]. Available: https://developer.amd.com/wordpress/media/2013/12/55723_SOG_Fam_17h_Processors_3.00.pdf
[19] ChampSim, 2020. [Online]. Available: https://github.com/ChampSim/ChampSim
[20] Standard Performance Evaluation Corporation, "SPEC CPU benchmark suites," 2020. [Online]. Available: https://www.spec.org/cpu/
[21] I. Cutress, "Examining Intel's ice lake processors: Taking a bite of the sunny cove microarchitecture," 2019. [Online]. Available: https://www.anandtech.com/show/14514/examining-intels-ice-lake-microarchitecture-and-sunny-cove/3
[22] A. Seznec, "The FNL+MML instruction cache prefetcher," *1st Instruction Prefetching Championship*, 2020.
[23] T. Nakamura, T. Koizumi, Y. Degawa, H. Irie, S. Sakai, and R. Shioya, "D-JOLT: Distant jolt prefetcher again," *1st Instruction Prefetching Championship*, 2020.
[24] A. Ros and A. Jimborean, "The entangling instruction prefetcher," *1st Instruction Prefetching Championship*, 2020.
[25] A. Seznec, "A 64-kbytes ITTAGE indirect branch predictor," *Championship Branch Prediction-3*, 2011.
[26] J. Kim, S. H. Pugsley, P. V. Gratz, A. L. N. Reddy, C. Wilkerson, and Z. Chishti, "Path confidence based lookahead prefetching," in *Proc. 49th Annu. IEEE/ACM Int. Symp. Microarchit.*, 2016, pp. 1–12.